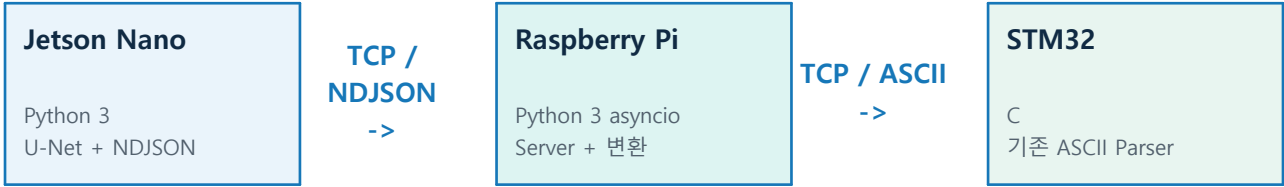


LeakGuard 분산 통신 시스템 설계안

Raspberry Pi Server - Jetson Nano Client - STM32 Client



NDJSON은 프로그래밍 언어가 아니며 Node.js를 요구하지 않습니다. Jetson Nano와 Raspberry Pi는 Python 3, STM32는 기존 C 코드를 유지하는 구성이 가장 적합합니다.

장치	권장 언어	통신 역할
Jetson Nano	Python 3	LeakStatus를 NDJSON으로 전송
Raspberry Pi	Python 3	TCP 서버, 검증, NDJSON/ASCII 변환
STM32	C	기존 ASCII 파서와 안전 FSM 유지

문서 버전 1.0 | 2026-07-27 | 팀 공유용

1. 핵심 아키텍처와 언어 선택

현재 Raspberry Pi - STM32 구간에서 ASCII 기반 C 파서를 개발하고 있으므로 이를 폐기하고 STM32에 JSON 파서를 새로 넣을 이유가 없습니다. Raspberry Pi가 두 형식 사이를 변환하면 장치별 장점을 유지할 수 있습니다.

권장안: Jetson -> Raspberry Pi는 TCP/NDJSON, Raspberry Pi -> STM32는 TCP/ASCII를 사용합니다. Raspberry Pi가 중앙 서버와 프로토콜 변환기를 겸합니다.

1.1 NDJSON의 의미

NDJSON(Newline-Delimited JSON)은 JSON 객체 하나를 한 줄에 하나씩 전송하고 줄바꿈 문자로 메시지 경계를 표시하는 형식입니다. Python, C, JavaScript, Java 등 어느 언어에서도 사용할 수 있습니다.

```
{"v":1,"type":"status","device_id":"jetson-01","seq":152}
{"v":1,"type":"heartbeat","device_id":"jetson-01","seq":153}
```

1.2 화면 문자열을 파싱하지 않는 이유

화면의 'Leak area: 2.364%' 같은 문구는 사람에게 보여주기 위한 표현입니다. 공백이나 문구가 변경되면 파서가 깨질 수 있습니다. Jetson의 LeakStatus 객체를 직접 JSON으로 직렬화해야 합니다.

```
LeakStatus object
-> json.dumps(status_message) + "\n"
-> Raspberry Pi TCP server
-> json.loads(line)
-> STM32 ASCII packet
```

1.3 장치별 책임

장치	주요 책임	장애 시 동작
Jetson Nano	U-Net 추론, 면적/확산/Danger ROI/4단계 판단, 최신 상태 전송	네트워크가 끊겨도 추론 지속, 자동 재접속
Raspberry Pi	다중 TCP 연결, 장치 등록, 검증, 변환, 라우팅, 로그	한 연결 오류가 서버 전체를 중단하지 않음
STM32	ASCII 파싱, 로컬 FSM, 액추에이터와 최종 안전 판단	통신 타임아웃 시 Fail-safe, DANGER latch 유지

2. 권장 메시지 프로토콜

2.1 연결 등록 - HELLO

모든 클라이언트는 연결 직후 역할, 장치 ID와 프로토콜 버전을 등록해야 합니다. 서버는 등록되지 않은 연결의 상태 메시지를 거부합니다.

```
{
  "v": 1,
  "type": "hello",
  "role": "jetson",
  "device_id": "jetson-01",
  "software_version": "1.0"
}
```

2.2 Jetson 상태 - STATUS

```
{
  "v": 1, "type": "status", "device_id": "jetson-01",
  "seq": 152, "timestamp": 1721800000.250,
  "detected": true, "leak_ratio": 2.364,
  "instant_ratio": 2.401, "spreading": false,
  "spreading_delta": 0.031,
  "danger_overlap": false, "danger_overlap_pixels": 0,
  "level": "WARNING", "leak_pixels": 4085,
  "inference_ms": 24.8, "fps": 18.5
}
```

필수 필드와 검증

필드	의미	서버 검증
v	프로토콜 버전	지원 버전인지 확인
type	hello/status/heartbeat	허용 메시지인지 확인
device_id	장치 고유 ID	등록된 연결과 일치
seq	순서 번호	중복 및 역순 거부
leak_ratio	Monitoring ROI 면적 비율(%)	0~100 범위
level	4단계 상태	허용된 4개 문자열
timestamp	상태 생성 시각	오래된 상태 TTL 검사

2.3 Raspberry Pi -> STM32 ASCII

```
LEAK,<seq>,<detected>,<ratio>,<spreading>,<danger>,<level>\n
LEAK,152,1,2.364,0,0,WARNING\n
HEARTBEAT,981,1721800000\n
CMD,381,RESET\n
ACK,381,RESET,OK\n
```

3. TCP 구현에서 놓치기 쉬운 부분

TCP는 메시지 단위가 아니라 연속된 바이트 스트림입니다. `recv()` 한 번이 메시지 하나라고 가정하면 안 됩니다.

3.1 수신 framing

- 수신 바이트를 연결별 buffer에 계속 누적합니다.
- 줄바꿈(`\n`)을 찾고 한 줄씩 분리합니다.
- 완전한 한 줄만 UTF-8 decode와 JSON parse를 수행합니다.
- 남은 불완전 데이터는 다음 `recv()`까지 보관합니다.
- 메시지 최대 크기를 예를 들어 4096 bytes로 제한합니다.

```
buffer += await reader.read(4096)
while b"\n" in buffer:
    line, buffer = buffer.split(b"\n", 1)
    message = json.loads(line.decode("utf-8"))
    validate_and_route(message)
```

3.2 전송 주기와 queue

데이터	권장 주기	정책
Jetson STATUS	5~10 Hz	최신 상태만 유지
Jetson HEARTBEAT	1 Hz	연결 생존 확인
단계 변경 / DANGER	즉시	일반 주기를 기다리지 않음
STM32 STATE	2~10 Hz	FSM 및 액추에이터 상태
CSV 일반 로그	1 Hz	단계 변경은 즉시 기록

상태 스트림은 오래된 값을 쌓지 않고 최신값으로 덮어씁니다. RESET 같은 명령과 단계 변경 이벤트는 별도의 제한된 queue에 넣고 ACK를 요구합니다.

3.3 재접속

구성 요소	권장 동작
Jetson	추론과 통신 thread 분리, 1/2/4/8초 backoff, 재연결 후 HELLO와 최신 상태 전송
Raspberry Pi	연결별 task 격리, 마지막 수신 시각 관리, 무응답 장치 OFFLINE 처리
STM32	1~2초 상태 미수신 시 COMM_ERROR 후보, 안전 출력으로 전환

4. 안전, 장애 대응과 보안

4.1 Raspberry Pi 단일 장애점

Raspberry Pi가 꺼지면 Jetson 상태가 STM32에 전달되지 않습니다. 따라서 STM32는 서버나 네트워크가 없어도 독립적으로 안전 상태로 전환해야 합니다.

STM32가 최종 안전 책임을 갖습니다. 통신 타임아웃 시 펌프 정지, 밸브 차단, 경고 출력 등 Fail-safe 를 수행하고 DANGER latch는 명시적인 안전 RESET 전까지 유지합니다.

선택 가능한 보조 안전 경로

```
Jetson -> Raspberry Pi TCP : monitoring / logging / remote control
Jetson -> STM32 UART      : emergency leak / danger signal
STM32      : final safety FSM and actuators
```

UART 보조 경로를 사용하지 않는 경우 Raspberry Pi 전원 차단, LAN 케이블 분리, 서버 프로세스 강제 종료 시험을 필수 항목으로 포함해야 합니다.

4.2 네트워크와 보안

- Raspberry Pi에는 고정 IP 또는 DHCP 예약을 사용합니다.
- 서버 IP, port, device_id는 설정 파일로 분리합니다.
- 서버를 인터넷에 직접 노출하지 않습니다.
- 메시지 종류, 길이, 수치 범위와 문자열 값을 검사합니다.
- 필요하면 장치별 공유 token 또는 HMAC을 추가합니다.
- Raspberry Pi와 Jetson 프로그램을 systemd 서비스로 등록합니다.

4.3 STM32 네트워크 인터페이스 결정

내장 Ethernet + lwIP, W5500, ESP32/ESP8266 중 어떤 방식을 사용하든지 확정해야 합니다. 선택에 따라 socket API, RTOS task, 수신 buffer와 재접속 구현이 달라집니다.

방식	고려사항
내장 Ethernet + lwIP	STM32 native socket, RTOS task 및 timeout 설계
W5500	하드웨어 socket 수, SPI 처리, 재접속 상태 관리
ESP 계열	UART 또는 AT 명령 계층, 모듈 재부팅과 Wi-Fi 재접속

5. 권장 구현 순서

단계	작업	완료 기준
1	프로토콜 v1 필드와 예제 확정	팀 전체가 동일 문서 승인
2	Raspberry Pi asyncio TCP 서버	다중 client 등록과 연결 관리
3	Python 가짜 Jetson/STM32 client	장비 없이 양방향 통신
4	NDJSON/ASCII 변환 테스트	분할, 결합, 오류 입력 통과
5	Jetson LeakStatus 연결	추론과 독립적으로 5~10Hz 전송
6	STM32 C TCP client 연결	기존 ASCII parser와 FSM 연동
7	장애 주입 시험	재부팅, cable 분리, timeout, 재접속
8	systemd와 로그	부팅 자동 실행과 이벤트 추적

5.1 권장 폴더 구조

```

Raspberry Pi/
  server.py           # asyncio TCP server
  config.yaml         # IP, port, timeout
  protocol.py         # NDJSON / ASCII codec
  client_registry.py  # role and device registry
  state_router.py     # Jetson -> STM32 routing
  event_logger.py
  tests/

Jetson Nano/leakguard/
  tcp_client.py
  protocol.py

STM32/
  tcp_client.c
  protocol_parser.c
  safety_fsm.c
  
```

5.2 통합 시험 체크리스트

확인	시험 항목
☐	Jetson과 STM32가 HELLO로 올바르게 등록된다.
☐	4단계 상태와 Danger ROI 이벤트가 STM32까지 전달된다.
☐	TCP 데이터가 분할되거나 합쳐져도 정상 파싱된다.
☐	잘못된 JSON, ASCII, 범위 초과 값이 거부된다.
☐	sequence 중복 및 역순 메시지가 거부된다.
☐	Jetson과 Raspberry Pi 재부팅 후 자동 재접속한다.
☐	STM32 통신 타임아웃과 Fail-safe가 동작한다.
☐	장시간 실행 시 queue와 메모리가 계속 증가하지 않는다.
☐	연결, 해제, 경고 단계 변경이 로그에 기록된다.

다음 즉시 작업: Raspberry Pi asyncio 서버와 Python 가짜 Jetson, 가짜 STM32 클라이언트를 먼저 구현해 프로토콜을 장비 없이 검증합니다.

상세 텍스트 문서: docs/communication/LeakGuard_Communication_Architecture_KO.txt